

Álvarez Párraga Jeremy

Link del repositorio: <https://github.com/DeJere/Aplicaciones-distribuida-Alvarez-P-rraga-.git>

Documentación Técnica - RegistroDistribuido

Sistema de Registro Distribuido con Replicación, Relojes de Lamport, Heartbeats y Elección de Coordinador.

1. Introducción

RegistroDistribuido es un prototipo de sistema distribuido compuesto por tres nodos réplica que se comunican mediante sockets TCP. El sistema implementa relojes lógicos de Lamport para mantener el orden de eventos, heartbeats para detección de fallos, el algoritmo Bully para elección de coordinador y autenticación basada en tokens.

2. Objetivos

- Implementar comunicación distribuida mediante TCP.
- Mantener consistencia básica entre réplicas.
- Detectar fallos mediante heartbeats.
- Elegir automáticamente un coordinador usando Bully.
- Registrar eventos con orden lógico utilizando relojes de Lamport.

3. Arquitectura General

El clúster está formado por tres nodos: N1 (9001), N2 (9002) y N3 (9003). Cada nodo mantiene un reloj lógico, una lista de pares activos, información del coordinador y un registro ordenado de eventos.

4. Componentes del Sistema

utils.py: utilidades compartidas.

nodo.py: lógica principal de los nodos.

cliente.py: cliente interactivo para enviar operaciones.

demo.py: demostración automática del sistema.

5. Funcionalidades de utils.py

LamportClock: administra el reloj lógico.

encode_message(): serializa mensajes JSON con prefijo de longitud.

recv_message(): recibe mensajes completos.

is_valid_token(): valida tokens autorizados.

log(): genera registros con timestamp y reloj lógico.

6. Funcionalidades de nodo.py

Procesamiento de mensajes operacion, heartbeat, election, ok, coordinator y replicate.

Replicación de eventos entre nodos.

Monitoreo de heartbeats.

Detección de fallos y ejecución del algoritmo Bully.

Mantenimiento de orden total mediante Lamport.

7. Cliente Interactivo

Permite enviar operaciones a cualquier nodo usando autenticación por token. Incluye comandos op <texto>, status y exit.

8. Formato de Mensajes

Todos los mensajes utilizan framing de 4 bytes para longitud seguido de un cuerpo JSON. Los tipos soportados son OPERACION, HEARTBEAT, ELECTION, OK, COORDINATOR y REPLICATE.

9. Seguridad

Los tokens válidos son token-cliente-1, token-cliente-2 y token-admin. Las solicitudes inválidas son rechazadas con un mensaje error.

10. Flujo de Operación

El cliente envía una operación. El nodo valida el token, actualiza el reloj Lamport, registra el evento, replica la información a los demás nodos y responde con una confirmación.

11. Tolerancia a Fallos

Los nodos intercambian heartbeats periódicos. Si el coordinador deja de responder, se inicia automáticamente una nueva elección mediante el algoritmo Bully.

12. Ejecución

python nodo.py 1 9001

python cliente.py --nodo 1 --token token-cliente-1

op Prueba de conexion

exit

python demo.py

13. Resultados Esperados

Las operaciones válidas son replicadas, los eventos se ordenan mediante Lamport, los fallos son detectados y el coordinador puede ser reemplazado automáticamente.

14. Anexos

cliente.py

```
import sys
import os
import socket
import argparse

sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
from common.utils import encode_message, recv_message

CLUSTER = {
    1: ("127.0.0.1", 9001),
    2: ("127.0.0.1", 9002),
    3: ("127.0.0.1", 9003),
}

def send_request(host: str, port: int, payload: dict) -> dict | None:
    """Envía un mensaje y espera respuesta."""
    try:
        s = socket.create_connection((host, port), timeout=5)
        s.sendall(encode_message(payload))
        resp = recv_message(s)
        s.close()
        return resp
    except ConnectionRefusedError:
        print(f" X No se pudo conectar a {host}:{port}")
        return None
    except Exception as e:
        print(f" X Error: {e}")
        return None

def main():
    parser = argparse.ArgumentParser(description="Cliente
```

```

RegistroDistribuido")
    parser.add_argument("--nodo", type=int, default=1, choices=[1, 2,
3],
                        help="Nodo destino (1, 2 o 3)")
    parser.add_argument("--token", default="token-cliente-1",
                        help="Token de autenticación")
    args = parser.parse_args()

    host, port = CLUSTER[args.nodo]
    print(f"      Cliente RegistroDistribuido      ")
    print(f" → Conectado a N{args.nodo} ({host}:{port})")
    print(f" → Token: {args.token}")
    print(f" Comandos: op <texto> | status | exit\n")

    while True:
        try:
            line = input("cliente> ").strip()
        except (EOFError, KeyboardInterrupt):
            print("\nSaliendo...")
            break

        if not line:
            continue

        parts = line.split(maxsplit=1)
        cmd = parts[0].lower()

        if cmd == "exit":
            break

        elif cmd == "op":
            if len(parts) < 2:
                print("  Uso: op <descripcion de la operacion>")
                continue
            operacion = parts[1]
            resp = send_request(host, port, {
                "type": "OPERACION",
                "token": args.token,
                "operacion": operacion,
            })
            if resp:
                if resp.get("type") == "ACK":
                    print(f"  ✓ ACK de N{resp['node']}:
{resp['detalle']}")
                else:
                    print(f"  ✗ Respuesta: {resp}")

            elif cmd == "status":
                print(f"  — Estado del clúster —")
                for nid, (h, p) in CLUSTER.items():
                    try:
                        s = socket.create_connection((h, p), timeout=1)
                        s.close()
                        estado = "✓ ACTIVO"
                    except Exception:
                        estado = "✗ CAÍDO"

```

```

        print(f"      N{nid} ({h}:{p}) {estado}")

    else:
        print(f"  Comando desconocido: '{cmd}'")
        print(f"  Comandos válidos: op <texto> | status | exit")

if __name__ == "__main__":
    main()

```

nodo.py

```

import argparse
import socket
import threading
import signal
import sys
import os
from typing import Dict, Any

# Aseguramos que el paquete common sea localizable cuando se ejecuta
# desde nodo/
sys.path.insert(0, os.path.join(os.path.dirname(__file__), os.pardir))
from common.utils import LamportClock, encode_message, recv_message,
is_valid_token

class NodeServer:
    def __init__(self, node_id: int, host: str, port: int):
        self.node_id = node_id
        self.host = host
        self.port = port
        self.clock = LamportClock()
        self.operations: list[Dict[str, Any]] = []
        self.server_socket = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
        self.server_socket.setsockopt(socket.SOL_SOCKET,
socket.SO_REUSEADDR, 1)
        self.is_running = threading.Event()

    def start(self) -> None:
        self.server_socket.bind((self.host, self.port))
        self.server_socket.listen(5)
        self.is_running.set()
        print(f"Nodo {self.node_id} escuchando en
{self.host}:{self.port}")

        try:
            while self.is_running.is_set():
                client_socket, client_address =
self.server_socket.accept()
                thread = threading.Thread(
                    target=self.handle_client,
                    args=(client_socket, client_address),
                    daemon=True,

```

```

        )
        thread.start()
    except KeyboardInterrupt:
        pass
    finally:
        self.shutdown()

    def shutdown(self) -> None:
        self.is_running.clear()
        try:
            self.server_socket.close()
        except OSError:
            pass
        print(f"Nodo {self.node_id} detenido")

    def handle_client(self, client_socket: socket.socket,
client_address: tuple[str, int]) -> None:
        try:
            message = recv_message(client_socket)
            if not message:
                return

            token = message.get("token", "")
            if not is_valid_token(token):
                error = {"type": "ERROR", "detail": "Token inválido o
ausente"}

                client_socket.sendall(encode_message(error))
                return

            if message.get("type") != "OPERACION":
                error = {"type": "ERROR", "detail": "Tipo de mensaje
desconocido"}

                client_socket.sendall(encode_message(error))
                return

            operation = str(message.get("operacion", "")).strip()
            current_clock = self.clock.tick()
            record = {
                "node": self.node_id,
                "clock": current_clock,
                "operation": operation,
            }
            self.operations.append(record)

            response = {
                "type": "ACK",
                "node": self.node_id,
                "clock": current_clock,
                "detalle": f"Operación '{operation}' registrada con
LC={current_clock}",
            }
            client_socket.sendall(encode_message(response))
            print(f"[{self.node_id}] {client_address} -> {operation}
(LC={current_clock})")
        except Exception as exc:
            error = {"type": "ERROR", "detail": f"Error interno:
{exc}"}

```

```

        try:
            client_socket.sendall(encode_message(error))
        except OSError:
            pass
    finally:
        try:
            client_socket.close()
        except OSError:
            pass

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="Nodo de registro distribuido con reloj de Lamport y autenticación simple"
    )
    parser.add_argument("node_id", type=int, help="Identificador del nodo")
    parser.add_argument("port", type=int, help="Puerto TCP donde escuchará el nodo")
    parser.add_argument("--host", default="127.0.0.1", help="Dirección del host donde escuchar")
    return parser.parse_args()

def main() -> None:
    args = parse_args()
    server = NodeServer(args.node_id, args.host, args.port)

    def handle_signal(signum: int, frame: Any) -> None:
        server.shutdown()
        sys.exit(0)

    signal.signal(signal.SIGINT, handle_signal)
    signal.signal(signal.SIGTERM, handle_signal)

    server.start()

if __name__ == "__main__":
    main()

```

demo.py

```

import subprocess
import sys
import time
import os
import socket

sys.path.insert(0, os.path.dirname(__file__))
from common.utils import encode_message, recv_message

CLUSTER = {
    1: ("127.0.0.1", 9001),

```

```

    2: ("127.0.0.1", 9002),
    3: ("127.0.0.1", 9003),
}

TOKEN_VALIDO = "token-cliente-1"
TOKEN_INVALIDO = "token-falso"

def send(host, port, payload, timeout=4):
    try:
        s = socket.create_connection((host, port), timeout=timeout)
        s.sendall(encode_message(payload))
        r = recv_message(s)
        s.close()
        return r
    except Exception as e:
        return {"error": str(e)}

def titulo(texto):
    print(f"\n{'='*60}")
    print(f" {texto}")
    print(f"{'='*60}")

if __name__ == "__main__":
    titulo("DEMO - RegistroDistribuido")

    # — Levantar los 3 nodos —————
    titulo("1. Levantando los 3 nodos...")
    nodo_dir = os.path.join(os.path.dirname(__file__), "nodo")
    procs = {}
    for nid in [1, 2, 3]:
        p = subprocess.Popen(
            [sys.executable, "nodo.py", str(nid), str(9000 + nid)],
            cwd=nodo_dir,
        )
        procs[nid] = p
        print(f" → N{nid} PID={p.pid}")

    print(" Esperando que arranquen (5 s)...")
    time.sleep(5)

    # — Prueba de autenticación fallida —————
    titulo("2. Prueba de SEGURIDAD - token inválido")
    r = send("127.0.0.1", 9001, {
        "type": "OPERACION", "token": TOKEN_INVALIDO,
        "operacion": "Operacion con token falso"
    })
    print(f" Respuesta N1: {r}")

    # — Envío de operaciones legítimas —————
    titulo("3. Enviando operaciones con token válido")
    ops = [
        "CARGA: Paquete #001 recibido en bodega A",
        "DESPACHO: Paquete #002 enviado a destino",
        "INVENTARIO: Stock actualizado - 150 unidades",
    ]

```

```

        "CARGA: Paquete #003 recibido en bodega B",
        "ALERTA: Temperatura fuera de rango en zona C",
    ]
    for i, op in enumerate(ops):
        nid = (i % 3) + 1 # distribuir entre los 3 nodos
        r = send("127.0.0.1", 9000 + nid, {
            "type": "OPERACION", "token": TOKEN_VALIDO,
            "operacion": op
        })
        print(f" Op→N{nid}: {r.get('detalle', r) if r else 'sin respuesta'}")
        time.sleep(0.3)

    time.sleep(1)

    # — Simular caída del coordinador (N3) —————
    titulo("4. Simulando caída del COORDINADOR (N3)")
    procs[3].terminate()
    procs[3].wait()
    print(" N3 terminado. Esperando detección de fallo y nueva elección (8 s)...")
    time.sleep(8)

    # — Enviar más operaciones sin el coordinador —————
    titulo("5. Operaciones DESPUÉS de la caída de N3")
    for i, op in enumerate(["CARGA: Paquete #004 – post-fallo",
        "DESPACHO: Paquete #005 – post-fallo"]):
        nid = (i % 2) + 1 # solo N1 y N2
        r = send("127.0.0.1", 9000 + nid, {
            "type": "OPERACION", "token": TOKEN_VALIDO,
            "operacion": op
        })
        print(f" Op→N{nid}: {r.get('detalle', r) if r else 'sin respuesta'}")
        time.sleep(0.3)

    # — Fin —————
    titulo("6. Demo completada – revisa las consolas de los nodos")
    print(" El log de eventos con ORDEN TOTAL se muestra en cada nodo.")
    print(" Presiona Ctrl+C para terminar todos los procesos.\n")

    try:
        time.sleep(30)
    except KeyboardInterrupt:
        pass
    finally:
        for nid, p in procs.items():
            try:
                p.terminate()
            except Exception:
                pass
        print(" Todos los nodos detenidos.")

```

Imágenes de la ejecución

```
PS C:\Users\Labo_02\Downloads\exampol> python nodo/nodo.py 1 9001
Nodo 1 escuchando en 127.0.0.1:9001
[1] ('127.0.0.1', 61401) -> test (LC=1)
[1] ('127.0.0.1', 61447) -> Prueba de conexion (LC=2)
[1] ('127.0.0.1', 64585) -> nnnnn (LC=3)
[1] ('127.0.0.1', 64654) -> hhh (LC=4)
[1] ('127.0.0.1', 64782) -> dsas (LC=5)
[1] ('127.0.0.1', 59155) -> asajs (LC=6)
[1] ('127.0.0.1', 53301) -> jaja (LC=7)
█
```

```
PS C:\Users\Labo_02\Downloads\exampol> python cliente/cliente.py --nodo 2 --token token-falso

Cliente RegistroDistribuido

→ Conectado a N2 (127.0.0.1:9002)
→ Token: token-falso
Comandos: op <texto> | status | exit

cliente> op ashj
X No se pudo conectar a 127.0.0.1:9002
cliente>
Saliendo...
```

```
PS C:\Users\Labo_02\Downloads\exampol> python nodo/nodo.py 2 9002
Nodo 2 escuchando en 127.0.0.1:9002
[2] ('127.0.0.1', 54297) -> hola comit (LC=1)
█
```

```
PS C:\Users\Labo_02\Downloads\exampol> python cliente/cliente.py --nodo 1 --token token-cliente-1

Cliente RegistroDistribuido

→ Conectado a N1 (127.0.0.1:9001)
→ Token: token-cliente-1
Comandos: op <texto> | status | exit

cliente> op jaja
✓ ACK de N1: Operación 'jaja' registrada con LC=7
cliente>
Saliendo
```

Comit del Repositorio

The screenshot shows the GitHub repository page for 'Aplicaciones-distribuidas-Avarez-P-raga'. The repository is a Python project implementing a distributed system. The file browser on the left shows the directory structure: `main`, `__pycache__`, `cliente`, `common`, `nodo`, `ANALISIS.md`, `README.md`, `demo.py`, and `t`. The right pane shows the `README.md` file content, which describes the project as a distributed system using a Lamport clock for ordering events. It details the roles of the nodes (N1, N2) and the clients (cliente), and provides instructions for running the demo.

Registro Distribuido con Reloj de Lamport Este proyecto implementa un nodo de registro distribuido simple con: - Reloj de Lamport (LamportClock) para orden lógico de eventos. - Enmascado de mensajes con longitud fija para comunicación TCP. - Autenticación simple de clientes con tokens válidos. - Nodo servidor que acepta operaciones y responde con 'ACK' o 'ERROR'. - Cliente que envía operaciones al nodo. - Script de demostración que ejecuta un flujo de operaciones válido e inválido. ## Estructura de archivos - 'common/relaj.py' - utilidades comunes: reloj de Lamport, framing, auth. - 'nodo/nodo.py' - servidor TCP que procesa operaciones de clientes. - 'cliente/cliente.py' - cliente de ejemplo que envía operaciones. - 'demo.py' - demo completo que arranca el nodo y prueba mensajes. - 'ANALISIS.md' - respuestas teóricas sobre el sistema distribuido. - 'README.md' - instrucciones de uso. ## Requisitos - Python 3.8+ - No se requieren dependencias externas.